COURSE SETUP GUIDE

OpenXR in Unity

Set up once, then two workflows for the rest of the semester

Meta Quest 2 / 3 / 3S. Windows and macOS.

This course is OpenXR only — Unity's own vendor-neutral packages. Not the Meta XR All-in-One SDK, and not Building Blocks.

Semester 2, 2026

Guide version 1.0

Contents

This guide has two halves.

Part 1 is setup. You do it once, and then you leave it alone. It ends with a scene of your own running standalone on a headset.

Part 2 is workflows. These are the two ways you will test your work, week after week, for the rest of the course. You will come back to this half constantly.

Everything here assumes you have already worked through `Software_and_Frameworks.md` and the Unity + GitHub guide, that you have Unity 6.3 LTS with Android Build Support installed, and that you have a Unity project sitting inside a working repository. It assumes no prior XR experience at all.

> **One warning before you start**
> This course uses **OpenXR only** — Unity's own packages on the open standard. We do **not** use the Meta XR All-in-One SDK and we do **not** use Building Blocks. Most Quest tutorials you find online do use them. Those tutorials aren't wrong, they're just a different stack, and installing both at once will break your project in ways that are genuinely hard to diagnose. If a tutorial starts by importing the Meta XR SDK, close it.

Part 0 — Before you start

Prerequisites

You need all of these before section 1. Every one of them is covered in an earlier guide.

- Unity Hub installed, and **Unity 6.3 LTS** (`6000.3.x`) installed through it
- Unity installed with **Android Build Support**, including **OpenJDK** and **Android SDK & NDK Tools**
- A course repository with the supplied `.gitignore` already in place
- A Unity project inside that repository that opens without errors
- A Meta Quest 2, 3, or 3S, charged, with its controllers paired
- **A USB-C cable that carries data.** Charge-only cables look identical and will waste an afternoon. Everything in this guide is done tethered

The two workflows

There are two ways you will test your work, and both run identically on Windows and macOS. You need both — they answer different questions.

Workflow	What it's for	Speed	Headset
A — Build to the headset	The truth. Real performance, real tracking, real comfort.	Minutes	Required
B — XR Interaction Simulator	Fast iteration in the Editor with keyboard and mouse.	Seconds	Not required

A third option, **Quest Link**, exists on Windows only and is described briefly in section 8. We don't teach it, because these two cover everything and work on every machine on the course.

Part 1 — Setup

1 What OpenXR actually is

For most of VR's short history, every headset came with its own SDK. Code written for a Rift didn't run on a Vive. Vendors competed by locking developers in, and developers paid for it every time they wanted to ship somewhere new.

OpenXR is the industry's answer to that. It's an open standard, managed by the Khronos Group — the same people behind Vulkan and OpenGL — that defines a common language between an application and a headset. Your app says "give me the pose of the right hand" or "tell me when the trigger is pressed", and whatever runtime is underneath answers in the same way, whether that's a Quest, a Vive, a Valve Index, or a headset that doesn't exist yet.

The stack looks like this:

```
Your Unity scene
      ↓
XR Interaction Toolkit     ← grabbing, teleporting, UI. Unity's, vendor-neutral.
      ↓
Unity OpenXR Plugin        ← translates Unity's XR API into OpenXR calls
      ↓
OpenXR runtime             ← supplied by Meta on Quest, by Valve on Index, etc.
      ↓
The headset
```

You write against the top two layers. The bottom two swap out underneath you.

The Oculus XR Plugin. Older tutorials use a Unity package called the Oculus XR Plugin, which was the pre-OpenXR path for Quest development. It is deprecated. Don't install it — and treat the rest of any tutorial that recommends it with suspicion, since it predates the current toolchain entirely.

The Meta XR SDK and Building Blocks. Meta's own SDK is current, well maintained, and genuinely good; its Building Blocks drop a working camera rig or hand tracking into a scene in seconds, and most Quest tutorials online use them. We don't, for two reasons. What you learn on OpenXR and the XR Interaction Toolkit transfers to any headset, while Building Block prefab names transfer nowhere. And when a Building Block works you learn nothing from it, whereas the rig you assemble by hand in section 6 is one you can debug when it breaks.

> **Do not install both stacks**
> The Meta SDK and Unity's OpenXR packages both want to configure the same project settings, own the same camera, and drive the same input. Installing the Meta SDK "just to try a tutorial" and then removing it leaves settings behind. Symptoms are listed in Section 13.

2 Headset and computer preparation

The course headsets are already set up. Developer Mode is enabled on all of them, and they are ready to receive your builds. You do not need a Meta developer account, an organisation, or the Meta Horizon mobile app to work on this course.

There is one thing you do have to do, and it is per-computer:

Authorise your computer. Plug the headset into your machine with the USB-C data cable and put the headset on. A prompt appears *inside the headset* asking whether to allow USB debugging. Tick **Always allow from this computer** and accept.

You will not see this prompt on your monitor. It is in the headset. Students routinely sit staring at a failed

build for ten minutes with an unanswered dialogue floating in front of a headset on the desk beside them.

You will need to do this again on each new computer you work at, and sometimes again after a headset system update.

> **Using your own headset**
> If you're working on a personal Quest rather than a course one, you'll need to enable Developer Mode on it yourself first: create a Meta developer account, create an organisation on the developer dashboard, then turn on Developer Mode for your headset in the Meta Horizon mobile app and restart the headset. Links are in `Software_and_Frameworks.md`.

> **Checkpoint**
> Open Meta Quest Developer Hub (MQDH) and check that your headset appears as connected over USB. If it does, you're ready. If it doesn't, fix it now — nothing later in this guide will work until this does.

3 Switching the project to Android

The Quest runs Android. Before Unity can build anything for it, the project has to be switched over.

3.1 Switch platform

File → **Build Profiles** → in the **Platforms** list, select **Android** → **Switch Platform**.

This re-imports every asset in the project into Android-compatible formats. On an empty project it takes a moment. On a project full of textures it can take a long while. Do it early, while the project is still small.

Set **Texture Compression** to **ASTC**, which is what Quest hardware wants.

3.2 Player settings that matter

Edit → **Project Settings** → **Player**, with the Android tab selected.

Under **Other Settings**:

Setting	Value	Why
Company Name / **Product Name**	Anything sensible	These generate your package name — set them before your first build
Graphics APIs	**Vulkan**	The modern path on Quest. Remove OpenGLES3 if it's listed above Vulkan
Scripting Backend	**IL2CPP**	Required — Mono won't build for Quest
Target Architectures	**ARM64** only	Untick ARMv7. Quest hardware is 64-bit and builds are smaller without it
Minimum API Level	**Android 12L (API 32)**	Meta's floor for current Horizon OS
Active Input Handling	**Input System Package (New)**	The Interaction Toolkit needs it. Changing this restarts the Editor

Under **Resolution and Presentation**, tick **Optimized Frame Pacing**.

3.3 Quality settings

Edit → Project Settings → Quality. Standalone VR is a mobile GPU rendering two eyes at high refresh rate, and it will punish you for treating it like a desktop.

Pick a low or medium quality level as the Android default, and inside it keep shadows modest and anti-aliasing at **4x MSAA** (MSAA is comparatively cheap on this hardware and matters a lot for how solid edges look up close).

Don't over-tune this now. Get a build running first, then optimise against real numbers from a real device.

4 Installing the XR packages

All of these come from Unity's own package registry, inside the project. Nothing here is downloaded from a website or imported from the Asset Store.

Install in this order — each one expects the previous.

4.1 XR Plugin Management

Edit → Project Settings → XR Plug-in Management → Install XR Plugin Management.

This is the settings panel where you choose an XR backend per platform. It doesn't do anything by itself.

4.2 Unity OpenXR Plugin

Still in **XR Plug-in Management**, select the **Android** tab (the little robot icon) and tick **OpenXR**. Unity installs the OpenXR plugin for you.

You'll see a red warning icon appear next to it. That's expected — it's Project Validation telling you OpenXR isn't configured yet. Section 5 fixes it.

4.3 XR Interaction Toolkit

Window → Package Manager → Unity Registry → search **XR Interaction Toolkit** → **Install**.

Then, still in Package Manager with the toolkit selected, open the **Samples** tab and import:

- **Starter Assets** — this contains the default input action maps that bind physical controller buttons to named actions. You need it. Without it you have a camera rig with no input.
- **XR Interaction Simulator** — the fast-iteration workflow from section 11. Import it now while you're here.

4.4 Later additions

You don't need these yet. They arrive with the activities that use them, and the activity brief will tell you.

- **XR Hands** (`com.unity.xr.hands`) — hand tracking instead of controllers
- **AR Foundation** + **Unity OpenXR: Meta** — passthrough and mixed reality on Quest 3 and 3S

> **Commit now**
> Your `Packages/manifest.json` and `Packages/packages-lock.json` have changed. Those two files are how a different machine reconstructs exactly this package set, so they belong in Git. Commit before you go further.

5 Configuring OpenXR

Installing OpenXR isn't the same as configuring it. This section is where most setup failures actually happen, and it's worth going slowly.

5.1 Enable the Meta feature group

Edit → Project Settings → XR Plug-in Management → Android tab.

Under the OpenXR entry you'll find a list of **OpenXR Feature Groups**. Tick **Meta Quest Support**.

This is what tells the OpenXR plugin to produce a build the Quest runtime will accept — the correct Android manifest entries, the right permissions, the device declaration. Without it your app will build happily and then refuse to launch.

5.2 Interaction profiles

Edit → Project Settings → XR Plug-in Management → OpenXR → Android tab.

Find **Enabled Interaction Profiles** and add:

- **Oculus Touch Controller Profile**

An interaction profile is OpenXR's description of a specific physical controller — which buttons exist, where they are, what they're called. Your input actions bind to profiles. **If this list is empty, your controllers will track as objects in space but no button will do anything**, which is a maddening bug to debug from scratch.

If you're on Quest 3 or 3S you can add **Meta Quest Touch Plus Controller Profile** as well. Adding profiles you don't own is harmless.

5.3 Render mode

On the same page, set **Render Mode** to **Single Pass Instanced**.

VR renders the scene twice, once per eye. Single Pass Instanced does that in one pass instead of two and is close to free performance. Leave it on unless something specific breaks.

5.4 Project Validation

Edit → Project Settings → XR Plug-in Management → Project Validation.

This panel checks your project against everything OpenXR and the Meta feature group expect, and it is the single most useful diagnostic tool in this whole guide. Get into the habit of coming back here whenever something is strange.

Each issue has a **Fix** button. Most can be fixed automatically. Work top to bottom until the Android tab is clean.

A note on the two icons:

- 🔴 **Errors** — must be fixed. Your build will fail or misbehave.
- 🟠 **Warnings** — should usually be fixed, but read them. Some are recommendations about quality settings that you may have deliberately chosen otherwise.

> **Checkpoint**
> Android tab of Project Validation shows no red errors. If it does, don't continue — nothing in section 6 will work, and you'll waste time debugging a scene when the problem is in settings.

6 The known-good scene

Now you build the smallest possible scene that proves the whole stack works. It has a floor, a thing to look at, and a camera rig you can move your head inside.

It is deliberately trivial. That's the point — **when this scene fails, the failure is in your setup and nothing else**. Keep it in your project all semester. Every time something mysterious breaks in a real activity, run this scene first to find out whether the problem is your work or your machine.

You will also run this exact scene through both workflows in Part 2, so that you know what "working" looks like in each of them before you have real work on the line.

6.1 Create the scene

File → **New Scene** → choose the **Basic (URP)** template → save it as `KnownGood.unity` somewhere sensible, like `Assets/Scenes/`.

Delete the **Main Camera** that came with the template. The XR rig brings its own.

6.2 Build the XR rig

GameObject → **XR** → **XR Origin (VR)**.

Look at what that just created in the Hierarchy, because this is the part worth understanding:

```
XR Origin (XR Rig)          ← the player's position in the world. Move THIS to move the player.
└── Camera Offset           ← height compensation, managed for you
    ├── Main Camera         ← the headset. Its transform is driven by tracking — never set it yoursel
    ├── Left Controller     ← driven by the left controller's tracked pose
    └── Right Controller
```

The mental model that saves the most confusion later: **the camera's position is an output, not an input.** It reports where the player's head is. If you want to move the player, you move the XR Origin, and the head keeps its offset within it. Trying to set the camera's transform directly is the most common beginner mistake in XR, and it fails in a way that makes people motion sick.

An **XR Interaction Manager** object also appeared. It's the broker between things that can interact and things that can be interacted with. One per scene, and you can ignore it.

Select **XR Origin** and set **Tracking Origin Mode** to **Floor**. This means y = 0 is the player's real floor, and a 1.7-metre-tall object will look 1.7 metres tall. On **Device** mode, y = 0 is wherever their head happened to be, and your whole scene will feel like it's floating.

6.3 Wire up input

Create an empty GameObject, name it `Input Action Manager`, and add the **Input Action Manager** component to it.

In its **Action Assets** list, add one element and assign **XRI Default Input Actions** — it came from the Starter Assets sample you imported in 4.3, under `Assets/Samples/XR Interaction Toolkit/.../Starter Assets/`.

This component's only job is to enable those input actions at runtime. It is one component doing one small thing, and forgetting it produces a rig where everything tracks correctly and no button works — the same symptom as a missing interaction profile, which is why these two are worth remembering as a pair.

6.4 Something to stand on and something to look at

- **GameObject** → **3D Object** → **Plane**, position `(0, 0, 0)`. This is your floor.
- **GameObject** → **3D Object** → **Cube**, position `(0, 1.2, 2)`, scale `(0.3, 0.3, 0.3)`.

The cube is at roughly chest height, two metres in front of you. Its job is to give you something with an obvious real-world size, so that when you put the headset on you can immediately tell whether the scale of the world is right.

Give both a material with some colour if you like. Flat grey is harder to judge depth against than you'd expect.

6.5 Add the scene to the build

File → Build Profiles → Scene List → Add Open Scenes, and make sure `KnownGood` is ticked and at the top.

A scene that isn't in this list will not be in your build. "Black screen on launch" is very often just this.

Save the scene. Commit.

7 Next time: start from the VR template

Everything in sections 3 to 6 was you adding XR to a project that didn't have it. That's the version worth doing once, because it's the version where you can see what each setting is for.

For your next project, you don't have to do any of it. In **Unity Hub → New Project**, choose the **VR** template. It arrives with XR Plugin Management installed, OpenXR enabled for Android, the XR Interaction Toolkit and its Starter Assets already imported, and a sample scene with a working rig — the end state of Part 1, ready to go.

Two things to know about it:

Check the settings anyway. The template configures itself for a general OpenXR target, not specifically for Quest. Walk section 5 again — the **Meta Quest Support** feature group and the **Oculus Touch Controller Profile** in particular — and run **Project Validation** before you build. It takes two minutes and it's the difference between a build that runs and a black screen.

You still need Android. The template doesn't know you're targeting a headset that runs Android, so section 3 still applies: switch platform, then the player settings.

Use the template from here on. Sections 3 to 6 are now your reference for when something in it looks wrong, and for understanding what the template did on your behalf.

Part 1 is done. You have a configured project and a scene worth testing. Everything from here is about getting it in front of your eyes.

Part 2 — Workflows

8 Choosing a workflow

You now have two ways to see your work. They are not alternatives to each other — they are tools for different moments, and a good week uses both.

	A — On the headset	B — XR Interaction Simulator
Speed per iteration	Minutes	Seconds
Headset required	Yes	No
Real performance	Yes	No
Real comfort and scale	Yes	No
Real tracking and controllers	Yes	No
Windows and macOS	Yes	Yes

8.1 The weekly loop

1. **Build the thing in the simulator.** Logic, layout, interactions, does the button do what it should. This is where most of your hours go, and it needs no headset — which means you can work anywhere, without booking equipment.
2. **Confirm on the headset before you call it done.** Every session, not just at the end of a project. Scale, comfort, reachability, and framerate are all things the simulator cannot tell you, and all things that can invalidate an afternoon's work.

Work that has only ever run in the simulator isn't finished work. If you take one thing from Part 2, take that.

8.2 A note on Quest Link

On Windows, Meta's desktop software offers **Quest Link**, which runs your scene live in the headset when you press Play — no build step, real tracking. If you have a Windows machine and want to try it, install the Meta Horizon desktop app, connect by cable, set Meta Quest Link as the active OpenXR runtime, and enable OpenXR on the **Windows, Mac, Linux** tab of XR Plug-in Management as well as the Android tab (Play mode reads the desktop settings, not the Android ones — that's the step people miss).

We don't teach it, for two reasons: it doesn't exist on macOS, and its performance is a flattering lie, because your PC's graphics card is doing the rendering rather than the headset's chip. It can replace step 1 of the loop above. It cannot replace step 2.

9 Workflow A — Building and running on the headset

This is the ground truth. Everything else is an approximation of it.

9.1 Your first build

1. Connect the headset by USB-C and put it on briefly to confirm there's no pending authorisation prompt (section 2).
2. **File** → **Build Profiles** → **Android** → check your device is listed under **Run Device**. Hit the refresh button beside it if it isn't.
3. Click **Build and Run**.
4. Choose an output folder. **Put it outside your repository** — a folder called `Builds` next to the

project, not inside it. Builds are large, they change completely every time, and they must never be committed. The course `.gitignore` covers the usual locations, but the reliable habit is keeping them out of the repo entirely.

5. Wait.

9.2 What "wait" means

Your first build will take a long time. Ten to twenty minutes is normal, and on an older laptop it can be more. IL2CPP is converting your C# to C++ and then compiling it for ARM64, and there's no shortcut.

Later builds of the same project are much faster — typically one to three minutes — because most of that work is cached.

Plan around this. Get one build through early in the semester, so that the twenty-minute version is behind you rather than in front of you on a day you need to show someone something.

9.3 Finding your app on the headset

Build and Run launches the app for you. But once you take the headset off and go back to the home screen, your app is not with the ones you bought.

In the headset: open the **Library**, then find the source or category filter, and choose **Unknown Sources**. Everything you have side-loaded lives there, listed by the Product Name you set back in section 3.2 — which is why setting it to something recognisable mattered.

9.4 The repeat loop

Once the first build is through, your normal cycle is: change something in Unity → **Build and Run** → put the headset on. If your device is already connected and authorised, that's a couple of minutes.

If the headset is connected and Unity can't see it, the fix is almost always in Section 13 under *headset not detected*.

> **Checkpoint**
> Your cube is floating in front of you at roughly chest height, it stays put when you move your head, and both controllers appear where your hands are. If so, your entire setup is correct. This is the moment Part 1 was for.

10 Meta Quest Developer Hub

MQDH is Meta's desktop companion for the headset. It isn't a third workflow — it sits alongside both of them, and it runs on **Windows and macOS** alike.

You'll see it called **ODH** or **Oculus Developer Hub** in older documentation, and in Meta's own URLs. Same tool, older name.

Connect over USB. MQDH also offers ADB over Wi-Fi, and on campus it is not worth the trouble — the wireless here is unreliable enough that a dropped connection mid-deploy will cost you more time than the cable ever will.

10.1 What you'll use it for

Deploying builds. Drag an `.apk` onto the **Device Manager** and it installs. Useful when Unity's Build and Run is being uncooperative, and useful for handing a build to a teammate who doesn't have your project.

Mirroring the headset to your screen. Start casting and what the wearer sees appears in a window on your computer:

- Your tutor can see what you're seeing without wrestling the headset off your head

- Your team can watch a playtest and take notes at the same time
- You can spot in a second that the problem is a controller not tracking

Capturing video and screenshots. Recording straight from the device beats filming a screen with your phone, whenever you need to show what your work actually looks like.

Reading logs. When a build launches and immediately dies, the reason is in the log, and this is where you read it.

10.2 Set it up now, not later

Install MQDH, connect your headset, and confirm casting works — before you need it under pressure in a lab. It takes five minutes on a quiet afternoon.

11 Workflow B — The XR Interaction Simulator

The simulator gives you a virtual headset and two virtual controllers driven by your keyboard and mouse, running in the Editor's Play mode. No headset, no build, no cable. It works identically on Windows and macOS.

This is where most of your development time should go.

11.1 Setting it up

You imported the sample in section 4.3. Find it under `Assets/Samples/XR Interaction Toolkit/.../XR Interaction Simulator/`, and drag the simulator prefab into your scene.

That's the whole setup. Press **Play**.

11.2 Driving it

An on-screen panel appears showing the current controls and which device you're currently driving. Read it — it's the authoritative reference, and it updates as you change modes.

The broad shape of it:

- **Mouse** moves the head, so you look around
- **WASD** moves the rig through the world
- Modifier keys hand control over to the **left** or **right** controller, so mouse and keyboard drive that hand instead of your head
- While driving a controller, other keys press its trigger, grip, and buttons

Spend two minutes with the on-screen panel the first time. Once the "which thing am I currently driving" idea clicks, the rest is muscle memory.

11.3 What the simulator will not tell you

This section matters more than the rest of this workflow. The simulator is a model of a headset, and every model leaves things out. It is silent — not wrong, *silent* — about:

Performance. Your Editor is running on a desktop GPU. The Quest runs on a mobile chip and needs to hold 72 frames a second in stereo. A scene that runs beautifully in the simulator can be unusable on device, and the simulator will never warn you.

Comfort. Motion sickness is the single most common failure in student XR work, and it's invisible on a monitor. Snap turns, acceleration, moving the horizon, poorly placed UI — you cannot evaluate any of it here.

Scale. Objects look right on screen and wrong at real size, constantly. A doorway that looks fine in the Scene view can be intimidatingly small when you're standing in it.

Real tracking. Real hands move differently to a mouse. Real controllers occlude, drift, and leave

tracking volume. Real people can't reach behind themselves.

Reachability and ergonomics. Whether the player can comfortably grab the thing you want them to grab, in the position a real body is actually in.

> Use the simulator to find out **whether it works**. Use the headset to find out **whether it's good**. Those are different questions.

12 Working across two machines

Most of you will work on a lab PC some days and your own machine on others. The Unity + GitHub guide covers moving a project between computers; this is the XR-specific part of that.

12.1 What has to match

Unity version	Exactly `6000.3.x`, the same as everyone else. Opening a project in a newer Editor upgrades it irreversibly for whoever opens it next
Package versions	Handled for you, as long as `Packages/manifest.json` and `Packages/packages-lock.json` are committed
Project settings	Committed, as long as `ProjectSettings/` is in the repo — it is, in the supplied `.gitignore`

12.2 What is per-machine and doesn't travel

None of this is in Git, and none of it should be. Expect to redo it the first time you sit at a new machine:

- Signing in to Unity Hub and applying your licence
- Installing and signing in to MQDH
- Authorising USB debugging, which is per computer *and* per headset (section 2)

12.3 What must never be committed

- `Library/`, `Temp/`, `Logs/`, `obj/` — Unity regenerates all of it
- **Builds and `.apk` files** — large, binary, and completely disposable. Keep your build output folder outside the repository entirely
- Anything under `Assets/Samples/` that you haven't modified is fine to commit, and simplest to leave in — it's small, and it keeps everyone's input actions identical

12.4 The habit that protects you

Make a device build at the end of every session, whichever machine you're on. It costs two minutes, it confirms the project still builds from a clean-ish state, and it means you find out that Wednesday's work doesn't hold framerate on Wednesday.

Part 3 — When it goes wrong

13 Troubleshooting

One symptom per heading. Find yours.

13.1 Setup and build

Unity says "no Android SDK found" or "Android SDK not found at path" The Android modules aren't installed, or Unity has lost track of them. Unity Hub → **Installs** → three-dot menu on 6.3 → **Add modules** → confirm **Android Build Support**, **OpenJDK**, and **Android SDK & NDK Tools** are all ticked. If they are, check **Edit** → **Preferences** → **External Tools** and re-tick the "use recommended version" boxes for SDK, NDK, and JDK.

The build fails with errors mentioning IL2CPP or ARMv7 Check section 3.2: **Scripting Backend** must be IL2CPP and **Target Architectures** must be ARM64 with ARMv7 unticked.

Project Validation shows red errors I don't understand Press **Fix**. That's what it's there for. If a fix doesn't stick, note the exact wording and bring it to your tutor — a persistent validation error usually means two packages are disagreeing, which is worth a second pair of eyes.

13.2 Device connection

Unity or MQDH can't see the headset Work through these in order: 1. Is the cable a **data** cable? Charge-only USB-C cables are extremely common and look identical. Try another. 2. Put the headset on and look for the USB debugging prompt (section 2). 3. Is the headset awake? A sleeping headset drops off the connection. 4. Try a different USB port — prefer one directly on the machine over a hub or dock. 5. Restart the headset.

It was working, and now Unity's Run Device list is empty Usually the authorisation was revoked by a headset update. Unplug, replug, put the headset on, accept the prompt again.

13.3 On the headset

The app builds successfully but I can't find it on the headset It's under **Library** → **Unknown Sources** (section 9.3), listed by Product Name, not by project name.

Black screen when the app launches In rough order of likelihood: 1. Your scene isn't in the **Scene List** in Build Profiles (section 6.5) 2. **Meta Quest Support** isn't ticked in the OpenXR feature groups (section 5.1) 3. OpenXR isn't ticked at all on the **Android** tab of XR Plug-in Management 4. Something threw an exception at startup — read the log in MQDH

The app runs but the controllers don't do anything Two causes, and they produce an identical symptom, so check both: 1. **Enabled Interaction Profiles** is empty (section 5.2) 2. The **Input Action Manager** is missing, or has no action asset assigned (section 6.3)

The controllers don't appear at all, or float in the wrong place Confirm they're paired and charged. If they track in the headset's home environment but not in your app, it's an interaction profile problem — see above.

Everything is the wrong size, or I'm standing underneath the floor Tracking Origin Mode is set to **Device** instead of **Floor** on the XR Origin (section 6.2).

The scene moves when I move my head, and it makes me feel ill Something is writing to the Main Camera's transform. Nothing in your code should ever do that — move the XR Origin instead (section 6.2).

13.4 Simulator

The simulator prefab is in the scene but nothing responds Check that **Active Input Handling** is set to **Input System Package (New)** (section 3.2). Also confirm the Game view has focus — the simulator only receives input when the Game view is the focused window. Click into it once.

I can look around but the controllers never move You haven't taken control of a hand. Check the on-screen control panel for the modifier keys that switch which device you're driving (section 11.2).

13.5 The one that's hardest to spot

Strange, contradictory errors after following an online tutorial If you installed the Meta XR All-in-One SDK at any point — even briefly, even if you removed it — you may have two XR stacks fighting. Signs: duplicate or unfamiliar entries in XR Plug-in Management, an `OVRManager` or `OVRCameraRig` anywhere in the project, Project Validation errors that reappear after you fix them, or compile errors mentioning `OVR` or `Oculus`.

The reliable fix is to remove the Meta SDK completely, delete the `Library` folder, and reopen the project — and if it's still confused, to check out a clean copy of the project from Git and reapply your work. This is a strong argument for committing often.

14 Where to get help

Start here, in this order:

1. **Project Validation** — `XR Plug-in Management → Project Validation`. Genuinely fixes a large share of problems by itself
2. **Section 13** above
3. **`Software_and_Frameworks.md`** — every version number, package name, and official documentation link for the whole stack lives there, and it's the authority if anything in this guide contradicts it
4. **The known-good scene** from section 6 — run it. It tells you instantly whether the problem is your work or your setup, and that halves the search space
5. **Your tutor** — bring the exact error text and what you've already tried

Official documentation is linked from `Software_and_Frameworks.md` rather than repeated here, so that there's one place to keep it current: Unity's OpenXR Plugin and XR Interaction Toolkit manuals, and Meta's device and MQDH documentation.

A note on searching for help. Most Quest tutorials and most Stack Overflow answers assume the Meta XR SDK. When you search, add `OpenXR` and `XR Interaction Toolkit` to your query, and be suspicious of any answer that tells you to install something from the Asset Store. If an answer's first step is "add a Building Block", it's solving a different problem than the one you have.

If a setup problem isn't answered anywhere in these three guides, that's a gap in the guides — tell your tutor so it can be fixed for everyone.